3F8: Inference: Short Lab Report

Zihe Liu

February 26, 2026

Abstract

This is the abstract.
Try for 1-2 sentences on each of: motive (what it's about), method (what was done), key results and conclusions (the main outcomes).

- Don't exceed 3 sentences on any one.
- Write this last, when you know what it should say!

Introduction

1. What is the problem and why is it interesting?
2. What novel follow-up will the rest of your report present?

Exercise a): Gradients of Log-Likelihood

In this exercise we consider the logistic classification model (aka logistic regression) and derive the gradient of the log-likelihood with respect to the parameter vector $\boldsymbol{\beta}$, given a vector of binary labels $\mathbf{y}$ and a matrix of input features $\mathbf{X}$. The probability of a positive class label is given by the logistic (sigmoid) function $\sigma(a) = 1/(1+e^{-a})$ applied to the activations:

$$P(y^{(n)} = 1 \mid \tilde{\mathbf{x}}^{(n)}) = \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}), \qquad P(y^{(n)} = 0 \mid \tilde{\mathbf{x}}^{(n)}) = 1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}) = \sigma(-\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}),$$

where $\tilde{\mathbf{x}}^{(n)} = (1, \mathbf{x}^{(n)})^T$ are the bias-augmented inputs, so that $\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)} = \beta_0 + \sum_{d=1}^{D} \beta_d x_d^{(n)}$. These two cases can be written compactly as a single Bernoulli likelihood:

$$P(y^{(n)} \mid \tilde{\mathbf{x}}^{(n)}, \boldsymbol{\beta}) = \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})^{y^{(n)}} \left(1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})\right)^{1-y^{(n)}}$$

Assuming N datapoints are independent and identically distributed, the likelihood of the full dataset is:

$$P(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}) = \prod_{n=1}^{N} \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})^{y^{(n)}} \left(1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})\right)^{1-y^{(n)}}$$

Giving the log-likelihood $\mathcal{L}(\boldsymbol{\beta}) = \log P(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta})$ of:

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{n=1}^{N} \left[y^{(n)} \log \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}) + (1 - y^{(n)}) \log \left(1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})\right) \right]$$

Note the derivative of the logistic function:

$$\frac{d\sigma(a)}{da} = \frac{e^{-a}}{(1+e^{-a})^2} = \frac{1}{1+e^{-a}} \cdot \frac{e^{-a}}{1+e^{-a}} = \sigma(a)\,(1 - \sigma(a))$$

Let $a^{(n)} = \boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}$, so that $\frac{\partial a^{(n)}}{\partial \boldsymbol{\beta}} = \tilde{\mathbf{x}}^{(n)}$. Then our derivatives are

$$\frac{\partial}{\partial \boldsymbol{\beta}} \left[y^{(n)} \log \sigma(a^{(n)}) \right] = y^{(n)} \cdot \frac{1}{\sigma(a^{(n)})} \cdot \underbrace{\frac{d\sigma}{da}}_{\sigma(1-\sigma)} \cdot \underbrace{\frac{\partial a^{(n)}}{\partial \boldsymbol{\beta}}}_{\tilde{\mathbf{x}}^{(n)}} = y^{(n)} \left(1 - \sigma(a^{(n)}) \right) \tilde{\mathbf{x}}^{(n)}$$

$$\frac{\partial}{\partial \boldsymbol{\beta}} \left[(1 - y^{(n)}) \log(1 - \sigma(a^{(n)})) \right] = (1 - y^{(n)}) \cdot \frac{-\sigma(a^{(n)})(1 - \sigma(a^{(n)}))}{1 - \sigma(a^{(n)})} \cdot \tilde{\mathbf{x}}^{(n)} = -(1 - y^{(n)}) \sigma(a^{(n)}) \tilde{\mathbf{x}}^{(n)}$$

Combining these two contributions for datapoint n:

$$y^{(n)}(1 - \sigma(a^{(n)}))\tilde{\mathbf{x}}^{(n)} - (1 - y^{(n)})\sigma(a^{(n)})\tilde{\mathbf{x}}^{(n)} = \left(y^{(n)} - \sigma(a^{(n)}) \right) \tilde{\mathbf{x}}^{(n)}.$$

Summing over all N datapoints,

$$\boxed{\frac{\partial \mathcal{L}(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = \sum_{n=1}^{N} \left(y^{(n)} - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}) \right) \tilde{\mathbf{x}}^{(n)} = \tilde{\mathbf{X}}^T (\mathbf{y} - \boldsymbol{\sigma}),}$$

where $\tilde{\mathbf{X}}$ is the $N \times (D+1)$ matrix whose n-th row is $(\tilde{\mathbf{x}}^{(n)})^T$, and $\boldsymbol{\sigma}$ is the N-dimensional vector with entries $\sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})$. The vectorised form follows because the sum $\sum_n (\cdot)\tilde{\mathbf{x}}^{(n)}$ is equivalent to the matrix product $\tilde{\mathbf{X}}^T$ applied to the residual vector $(\mathbf{y} - \boldsymbol{\sigma})$.

Exercise b) Vectorized Gradient Ascent

In this exercise we write vectorised pseudocode to estimate the parameters $\boldsymbol{\beta}$ using gradient ascent of the log-likelihood. The update rule is

$$\boldsymbol{\beta}^{(\text{new})} = \boldsymbol{\beta}^{(\text{old})} + \eta \left. \frac{\partial \mathcal{L}(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} \right|_{\boldsymbol{\beta} = \boldsymbol{\beta}^{(\text{old})}} = \boldsymbol{\beta}^{(\text{old})} + \eta \, \tilde{\mathbf{X}}^T \left(\mathbf{y} - \boldsymbol{\sigma}^{(\text{old})} \right),$$

where $\boldsymbol{\sigma}^{(\text{old})}$ is the vector of predictions $\sigma(\boldsymbol{\beta}^{(\text{old})T} \tilde{\mathbf{x}}^{(n)})$ under the current parameters. The pseudocode is as follows:

```
Function estimate_parameters:

   Input:  feature matrix X (N x D), labels y (N x 1),
           learning rate eta, number of iterations T
   Output: vector of coefficients b ((D+1) x 1)

   Code:
      X_tilde = [ones(N,1), X]        % augment with bias column
      b = randn(D+1, 1)               % initialise randomly

      for t = 1 to T:
          sigma = 1 / (1 + exp(-X_tilde * b))   % predictions (N x 1)
          grad = X_tilde^T * (y - sigma)         % gradient ((D+1) x 1)
          b = b + eta * grad                     % gradient ascent step

      return b
```

The learning rate η should be chosen small enough that the log-likelihood increases roughly monotonically, but large enough that convergence is achieved within a reasonable number of iterations. In practice, one can start with a small value (e.g. $\eta = 0.01$) and increase it until the learning curve begins to oscillate, then reduce slightly. The number of iterations T should be large enough for the log-likelihood to plateau.

Exercise c) Data Visualisation

The dataset consists of $N = 1000$ datapoints with two-dimensional input features $\mathbf{x}^{(n)} \in \mathbb{R}^2$ and binary class labels $y^{(n)} \in \{0, 1\}$. The dataset is visualised in Figure 1.

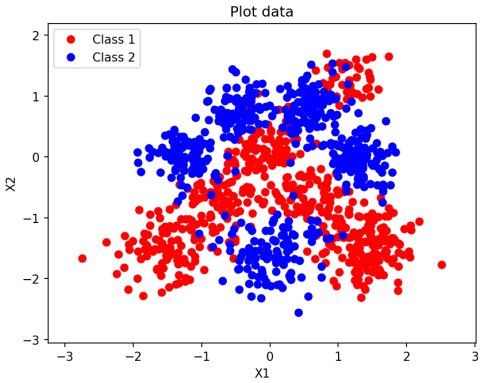

Figure 1: Visualisation of the dataset in the (X_1, X_2) input space. Red points are Class 1 ($y = 0$) and blue points are Class 2 ($y = 1$).

From Figure 1, we observe that Class 2 (blue) is concentrated in the upper and lower area of the input space, while Class 1 (red) is interspersed between the 2 blue regions. The two classes overlap significantly, particularly around the boundaries, and no single straight line can cleanly separate them. A linear classifier produces a decision boundary of the form $\boldsymbol{\beta}^T \tilde{\mathbf{x}} = 0$, which is a straight line in 2D. Given the non-linear arrangement of the classes, such a linear boundary would inevitably misclassify a substantial portion of the data. A non-linear decision boundary would be needed to capture the true class structure more accurately.

Exercise d) Training the Linear Classifier

The data is split randomly[1] into training ($N_{\text{train}} = 800$) and test ($N_{\text{test}} = 200$) sets. Gradient ascent is implemented as:

```
sigmoid_value = predict(X_tilde_train, w)
w = w + alpha * np.dot(X_tilde_train.T, y_train - sigmoid_value)
```

We train the classifier with learning rate $\eta = 0.0005$ for $T = 100$ iterations. The average log-likelihood on the training and test sets as the optimisation proceeds is shown in Figure 2.

Both curves increase monotonically and plateau after approximately 20 iterations, indicating that the optimisation has converged and $\eta = 0.0005$ is an appropriate learning rate. The training and test curves follow a similar trajectory, suggesting there is no overfitting – as expected for a model with only $D + 1 = 3$ parameters fitting 800 datapoints.

Figure 3 displays the contours of the predictive probabilities overlaid on the data. The contours are parallel straight lines, confirming the linear nature of the decision boundary. The 0.5 contour (the decision boundary) runs roughly diagonally, separating the upper region (higher $P(y = 1)$) from the lower region. However, as anticipated in Exercise c), a linear boundary cannot capture the non-linear class structure: many red (Class 1) and blue (Class 2) points lie on the wrong side of the decision boundary. This motivates the non-linear feature expansion in later exercises.

[1] A fixed numpy random seed of 1 was used for reproducibility.

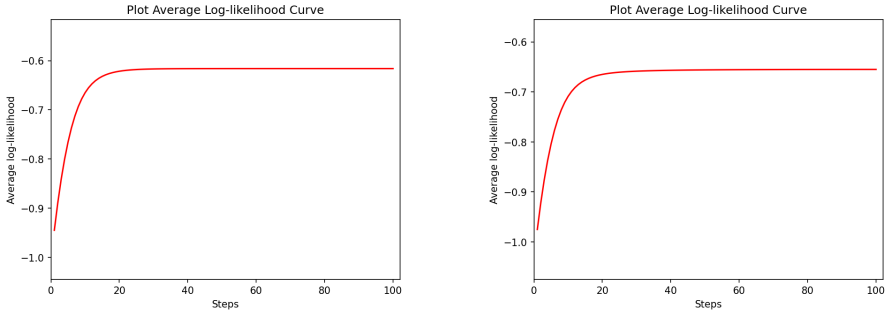

Figure 2: Learning curves showing the average log-likelihood per datapoint on the training (left) and test (right) datasets.

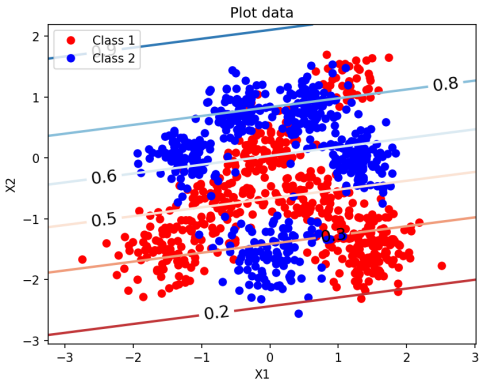

Figure 3: Contours of the class predictive probabilities for the linear logistic classifier.

Exercise e) Log-Likelihoods and Confusion Matrix

The final average training and test log-likelihoods are shown in Table 1. The test log-likelihood (-0.6550) is slightly lower than the training log-likelihood (-0.6161), which is expected since the model was fitted to the training data. The small gap between them confirms the absence of significant overfitting. However both values are close to $-\ln 2 \approx -0.693$, which is the log-likelihood of a random coin-flip classifier, indicating that the linear model offers only a modest improvement over random guessing on this non-linearly separable dataset.

The 2×2 confusion matrix on the test set (using threshold $\tau = 0.5$) is shown in Table 2. The classifier correctly identifies 71.88% of Class 1 ($y = 1$) points and 61.54% of Class 0 ($y = 0$) points. However, the error rates are substantial: 38.46% of true Class 0 points are misclassified as Class 1, and 28.12% of true Class 1 points are misclassified as Class 0. This confirms that the linear decision boundary is inadequate for this dataset.

Avg. Train ll	**Avg. Test ll**
-0.6161	-0.6550

Table 1: Average training and test log-likelihoods for the linear classifier.

		$\hat{y}$	
		0	1
y	0	0.6154	0.3846
	1	0.2812	0.7188

Table 2: Confusion matrix on the test set (fractions conditioned on true label).

Exercise f) Non-Linear Feature Expansion with RBFs

To overcome the limitations of the linear classifier, we expand the original two-dimensional inputs through a set of N_{train} Gaussian radial basis functions (RBFs), each centred on one training datapoint:

$$\tilde{\mathbf{x}}_{\text{RBF}}^{(n)} = \Big(1,\ \phi_1(\mathbf{x}^{(n)}),\ \phi_2(\mathbf{x}^{(n)}),\ \ldots,\ \phi_{N_{\text{train}}}(\mathbf{x}^{(n)})\Big)^T \in \mathbb{R}^{N_{\text{train}}+1}$$

where each basis function ϕ_m is a Gaussian kernel centred on the m-th training point $\mathbf{x}^{(m)}$:

$$\phi_m(\mathbf{x}^{(n)}) = \exp\left(-\frac{1}{2l^2}\sum_{d=1}^{D}\left(x_d^{(n)} - x_d^{(m)}\right)^2\right) = \exp\left(-\frac{\|\mathbf{x}^{(n)} - \mathbf{x}^{(m)}\|^2}{2l^2}\right)$$

The hyperparameter $l > 0$ is the *width* of the RBFs. Intuitively, $\phi_m(\mathbf{x}^{(n)})$ measures how close $\mathbf{x}^{(n)}$ is to the m-th training centre: it equals 1 when $\mathbf{x}^{(n)} = \mathbf{x}^{(m)}$ and decays to 0 as the Euclidean distance $\|\mathbf{x}^{(n)} - \mathbf{x}^{(m)}\|$ grows relative to l.

The logistic classifier is now applied in this $(N_{\text{train}}+1)$-dimensional feature space rather than the original $(D+1)$-dimensional space:

$$P(y^{(n)} = 1 \mid \mathbf{x}^{(n)}) = \sigma\Big(\boldsymbol{\beta}^T \tilde{\mathbf{x}}_{\text{RBF}}^{(n)}\Big) = \sigma\left(\beta_0 + \sum_{m=1}^{N_{\text{train}}} \beta_m\, \phi_m(\mathbf{x}^{(n)})\right).$$

Because the mapping $\mathbf{x} \mapsto (\phi_1(\mathbf{x}), \ldots, \phi_{N_{\text{train}}}(\mathbf{x}))$ is non-linear, the resulting decision boundary $\boldsymbol{\beta}^T \tilde{\mathbf{x}}_{\text{RBF}} = 0$ is a non-linear surface in the original input space.

Experimental Results

We train the logistic classifier with RBF features for three widths $l \in \{0.01, 0.1, 1\}$, using learning rates $\eta \in \{0.0001, 0.0005, 0.0001\}$ and $T = 500$ gradient ascent steps for each, respectively. The learning rates were chosen so that the log-likelihood increases roughly monotonically without oscillation; Figure 4 displays our results.

Discussion of results

For $l = 0.01$ (top row), the basis functions are extremely narrow and the contour plot shows highly localised, spiky predictive probabilities centred on individual training points, with no smooth interpolation between them. The training log-likelihood converges to -0.8017 and the test log-likelihood to -0.7015. Both values are close to $-\ln 2 \approx -0.693$, indicating poor performance: the features are so localised that the model essentially treats each training point independently and fails to learn any useful global structure. Interestingly the test log-likelihood is slightly better than the training log-likelihood, which occurs because the near-identity feature matrix makes gradient ascent slow on the 800 training parameters, while the sparse test features produce predictions close to 0.5 (the log-likelihood of a coin flip), and the model has not yet had enough iterations to memorise the training data.

For $l = 0.1$ (middle row), the basis functions overlap sufficiently for the classifier to learn a meaningful non-linear decision boundary. The contour plot shows structured probability regions that follow the true class distribution – high $P(y = 1)$ in the upper and lower regions where Class 2 (blue) points concentrate, and high $P(y = 0)$ in the central band where Class 1 (red) points dominate. The learning curves are still rising at 500 steps, indicating that the optimisation has not fully converged and additional iterations (or a larger learning rate) could further improve performance. The final train and test log-likelihoods are -0.3361 and -0.3914, respectively – a substantial improvement over both the linear classifier and $l = 0.01$.

For $l = 1.0$ (bottom row), the broad basis functions produce smooth contours that capture the large-scale class structure well. The contour plot shows an elliptical decision boundary roughly separating the two

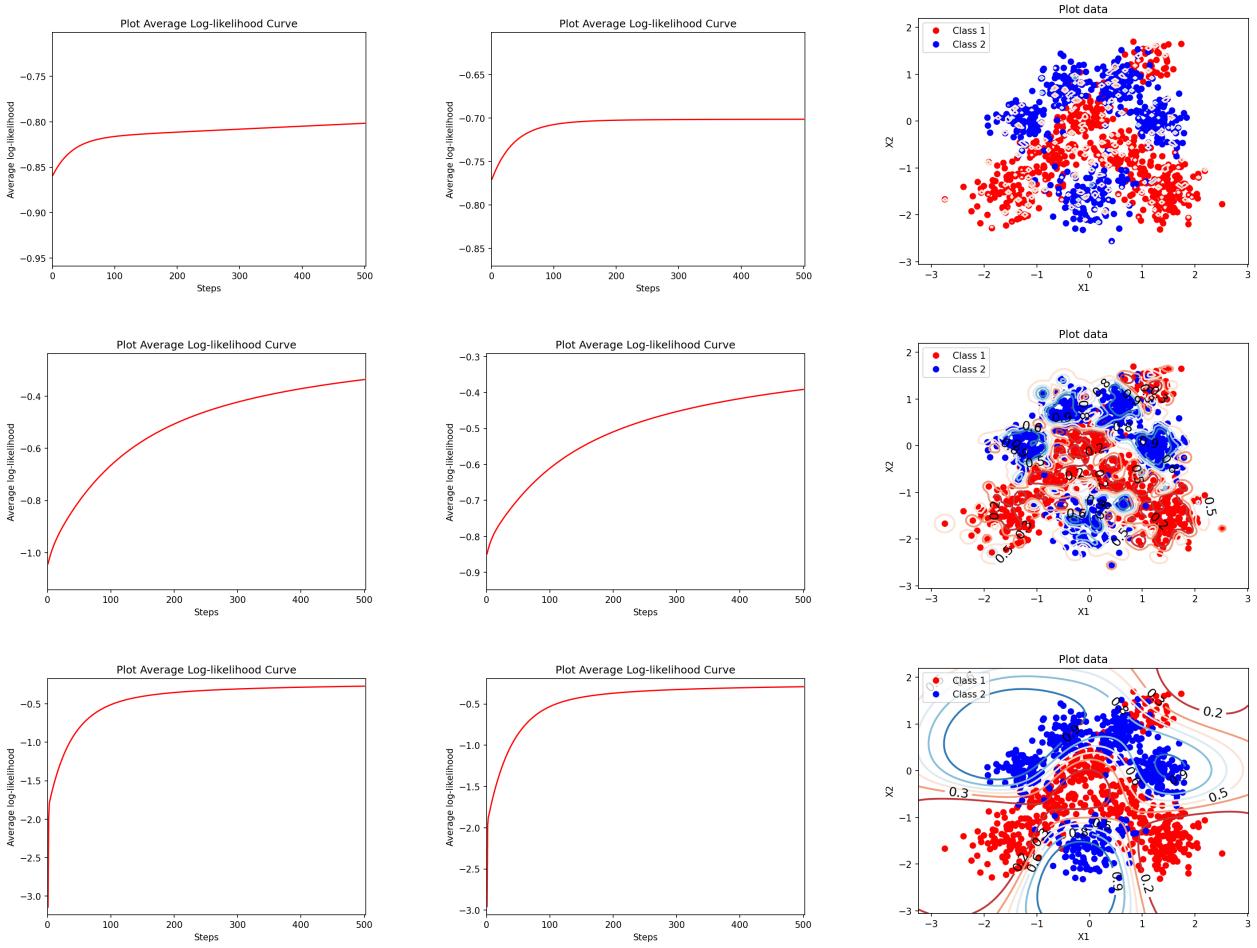

Figure 4: Training log-likelihood (left), test log-likelihood (middle), and predictive probability contours (right) for the RBF-expanded logistic classifier. **Top row:** $l = 0.01$, $\eta = 0.0001$, $T = 500$. **Middle row:** $l = 0.1$, $\eta = 0.0005$, $T = 500$. **Bottom row:** $l = 1.0$, $\eta = 0.0001$, $T = 500$.

classes. The learning curves also suggest the optimisation has not fully converged. The final train and test log-likelihoods are -0.2701 and -0.2900, respectively – the best among the three settings. The small gap between training and test log-likelihoods indicates that the smooth model generalises well without overfitting.

Overall, increasing l from 0.01 to 1.0 transitions the model from overly localised (underfitting due to feature sparsity) through well-structured non-linear boundaries to smooth, well-generalising predictions. The learning rates have not been exhaustively tuned, and particularly for $l = 0.1$ and $l = 1.0$ the learning curves suggest that convergence has not been fully reached, so further tuning of η and T could improve results.

Role of the width l

The width l controls the *smoothness* of the feature expansion and thereby governs the bias–variance trade-off:

- **Small l (e.g. $l = 0.01$):** Each basis function ϕ_m is extremely narrow – it is essentially non-zero only very close to its centre $\mathbf{x}^{(m)}$. The feature matrix becomes approximately the identity (each training point activates only its own basis function), so the model memorises the training labels but cannot generalise to unseen points that lie between training centres. This leads to *overfitting*: strong training performance but poor test performance.

Avg. Train ll	Avg. Test ll
-	-

Table 3: Results for $l = 0.01$

Avg. Train ll	Avg. Test ll
-	-

Table 4: Results for $l = 0.1$

Avg. Train ll	Avg. Test ll
-	-

Table 5: Results for $l = 1$

		$\hat{y}$	
		0	1
y	0	-	-
	1	-	-

Table 6: Conf. matrix $l = 0.01$.

		$\hat{y}$	
		0	1
y	0	-	-
	1	-	-

Table 7: Conf. matrix $l = 0.1$.

		$\hat{y}$	
		0	1
y	0	-	-
	1	-	-

Table 8: Conf. matrix $l = 1$.

- **Large l (e.g. $l = 1$):** Each basis function ϕ_m is broad and responds substantially to many datapoints. The N_{train} features become highly correlated (approaching a constant vector as $l \to \infty$), so the effective dimensionality of the feature space is reduced. The model captures smooth, large-scale structure in the data. If l is *too* large the model loses the ability to capture fine-grained class boundaries and *underfits*.
- **Intermediate l (e.g. $l = 0.1$):** The basis functions overlap sufficiently for neighbouring training points to share information, yet remain localised enough to capture non-linear class structure. This typically yields the best trade-off between fitting the training data and generalising to the test data.

Impact on the learning rate η

For small l, the feature matrix $\tilde{\mathbf{X}}_{\text{RBF}}$ is approximately the identity, but the gradients have limited magnitude due to sparse activations, so a moderate learning rate suffices. For intermediate l, the features are more correlated and the gradient magnitudes grow (each training point contributes to many basis functions simultaneously), so a larger learning rate can be used to speed convergence without instability. For large l, the feature matrix is near-singular with very large feature values (all basis functions are close to 1), producing large gradient magnitudes that demand a smaller learning rate to avoid divergence.

1 Exercise g)

The final final training and test log-likelihoods per datapoint obtained for each setting of $l = \{0.01, 0.1, 1\}$ are shown in tables 3, 4 and 5. These results indicate that... The 2×2 confusion matrices for the three models trained with $l = \{0.01, 0.1, 1\}$ are show in tables 6, 7 and 8. After analysing these matrices, we can say that... When we compare these results to those obtained using the original inputs we conclude that...

2 Conclusions

1. Draw together the most important results and their consequences.
2. List any reservations or limitations.